Entrega 1 — Diseño del esquema NoSQL y plan de implementación

Nov 1, 2026 · @Miguel

1. Contexto y objetivo

Vamos a almacenar los límites de los municipios PDET y dos datasets abiertos de huellas de edificaciones en MongoDB, y a usar sus índices espaciales 2dsphere para contar los techos y sumar su área en cada municipio PDET. Esta entrega define la metodología y el modelo de datos sobre los que se construyen las entregas 2 a 5.

La Oficina de Investigación e Información de la UPME necesita una forma reproducible de ordenar los municipios PDET según el área de techos apta para paneles solares. A mayor área de techos, mayor potencial solar. El proyecto debe usar una solución NoSQL y comparar al menos dos datasets abiertos de edificaciones.

Ítem	Valor
Curso	Administración de Bases de Datos — Proyecto final (Prof. Andrés Oswaldo Calderón Romero, Ph.D.)
Integrantes	Derek Sarmiento, José Medina, Miguel Duarte, Sebastián Sánchez Vásquez, Samuel Emperador
Repositorio	https://github.com/dba-upme-pdet/techos-solares-pdet
Fecha de entrega	Semana 1 — lunes 2 de noviembre de 2026, antes de las 2:00 PM (GitHub + BrightSpace)

Alcance de la Entrega 1: (a) plan de implementación, (b) diagrama E-R y (c) diccionario de datos. En esta entrega todavía no se cargan datos; la carga empieza en la Entrega 2.

2. Decisiones tecnológicas

Elegimos MongoDB (Community Server 7.x, en Docker) como motor NoSQL, y Microsoft Building Footprints y Google Open Buildings v3 como los dos datasets de edificaciones.

2.1 Motor NoSQL

MongoDB guarda geometrías de forma nativa en GeoJSON, las indexa con `2dsphere` y resuelve consultas de punto en polígono e intersección con `$geoWithin` y `$geoIntersects`. Su pipeline de agregación permite contar y sumar por municipio sin salir de la base de datos.

Opción	Tipos geográficos nativos	Índice espacial	Agregación por polígono	Decisión
MongoDB	GeoJSON Point, Polygon, MultiPolygon	2dsphere	`$geoWithin` + `$group` / `$lookup`	**Seleccionado**
Elasticsearch	geo_point, geo_shape	Árbol BKD	Consulta geo_shape + agregaciones	Alternativa de respaldo; más pesado de operar
Apache Cassandra	Ninguno (requiere GeoMesa)	Externo	Débil	Descartado
Neo4j	Solo puntos	Índice de puntos	No soporta polígonos	Descartado

MongoDB no calcula áreas geodésicas. El área de cada huella se calcula en Python al momento de la carga y se guarda como un campo.

2.2 Datasets de edificaciones

Dataset	Formato para Colombia	Atributos clave	Rol
Microsoft Building Footprints	GeoJSON Lines, particionado por quadkey (archivo índice `dataset-links.csv`)	polígono, `confidence`, `height` (–1 cuando no se conoce)	Fuente A

Dataset	Formato para Colombia	Atributos clave	Rol
Google Open Buildings v3	CSV.gz, particionado por celda S2	polígono (WKT), `area_in_meters`, `confidence`, `full_plus_code`	Fuente B
GlobalBuildingAtlas (TUM)	Teselas 3D LoD1	polígono + altura	No se usa; queda como posible tercera fuente

Las dos fuentes elegidas entregan polígonos 2D para toda Colombia y un puntaje de confianza, así que se pueden comparar en igualdad de condiciones. El campo `area_in_meters` de Google también sirve para validar el área que calculamos.

2.3 Límites administrativos

Los polígonos municipales vienen del Marco Geoestadístico Nacional del DANE (MGN 2025, nivel municipal). La lista PDET viene del archivo `MunicipiosPDET.xlsx` que entrega el curso en la carpeta `resources`: 170 municipios en 16 subregiones (Decreto 893 de 2017, Agencia de Renovación del Territorio). Las dos fuentes se cruzan por el código DIVIPOLA de 5 dígitos.

3. Plan de implementación

El flujo de trabajo tiene cinco fases, una por entrega, y cada paso es un script de Python numerado dentro del repositorio para que cualquiera pueda volver a correrlo de principio a fin.

1. **Fase 0 — Entorno (Entrega 1).**
   - MongoDB 7 en Docker (`docker-compose.yml`), Python 3.11 con `pymongo`, `geopandas`, `shapely`, `pyproj`, `pandas`.
   - La configuración (rutas, URLs, umbrales) vive en un solo archivo `config.yaml`; ningún script tiene valores escritos a mano.
2. **Fase 1 — Límites PDET (Entrega 2).**
   - Descargar el ZIP del MGN 2025 del DANE y extraer la capa municipal; leer la lista PDET de `MunicipiosPDET.xlsx`.

- Completar con cero a la izquierda los códigos del Excel (vienen como números, ej. 5031 → 05031), cruzar por código DIVIPOLA de 5 dígitos y quedarse con los 170 municipios PDET; registrar cualquier código que no cruce.
- Reproyectar de MAGNA-SIRGAS (EPSG:4686) a WGS84 (EPSG:4326), el único sistema de referencia que acepta `2dsphere`.
- Reparar geometrías inválidas (`make_valid`), revisar la orientación de los anillos y que cada documento quede por debajo del límite de 16 MB de MongoDB.
- Cargar en `municipalities` y crear el índice `2dsphere` sobre `geometry`.

3. **Fase 2 — Huellas de edificaciones (Entrega 3).**
   - Descargar solo los archivos por quadkey de Microsoft y por celda S2 de Google que intersecten los rectángulos envolventes de los municipios PDET.
   - Leer cada archivo por bloques; descartar las edificaciones fuera de esos rectángulos antes de insertarlas.
   - Por edificación: validar el polígono, calcular su área geodésica en m² (`pyproj.Geod`) y su centroide.
   - Inserción masiva (`insert_many`, no ordenada, lotes de 10.000) en `buildings`; crear los índices después de la carga, no antes.
   - Registrar cada carga (archivo, filas leídas, cargadas, rechazadas, duración) en `ingestion_runs`; hacer el análisis exploratorio (EDA).
4. **Fase 3 — Conteo y área de techos (Entrega 4).**
   - Para cada municipio PDET, seleccionar las edificaciones cuyo centroide cae `$geoWithin` su polígono, por fuente.
   - Asignar por centroide pone cada edificación en un solo municipio, así que las que quedan en el borde no se cuentan dos veces.
   - Aplicar los filtros de calidad (confianza mínima, área mínima) definidos en `config.yaml`.
   - `$group` para obtener número de edificaciones, área total y área promedio; guardar en `rooftop_stats`; comparar las dos fuentes.
   - Exportar tablas (CSV) y mapas coropléticos por municipio y subregión.
5. **Fase 4 — Informe técnico (Entrega 5).** Metodología, resultados, comparación de fuentes, ranking de municipios PDET y recomendaciones para la UPME.

Umbrales pendientes, se fijan después del EDA: área mínima por edificación (candidato: 20 m²) y confianza mínima (candidato: 0,70 para Google; el campo de Microsoft suele venir vacío).

4. Diseño del esquema y modelado de datos

El modelo tiene seis entidades, cada una guardada como una colección de MongoDB; BUILDING es la entidad central y de mayor volumen, y se asocia a un municipio según la ubicación de su centroide.

Seis colecciones; cada edificación se asocia a un municipio por su centroide

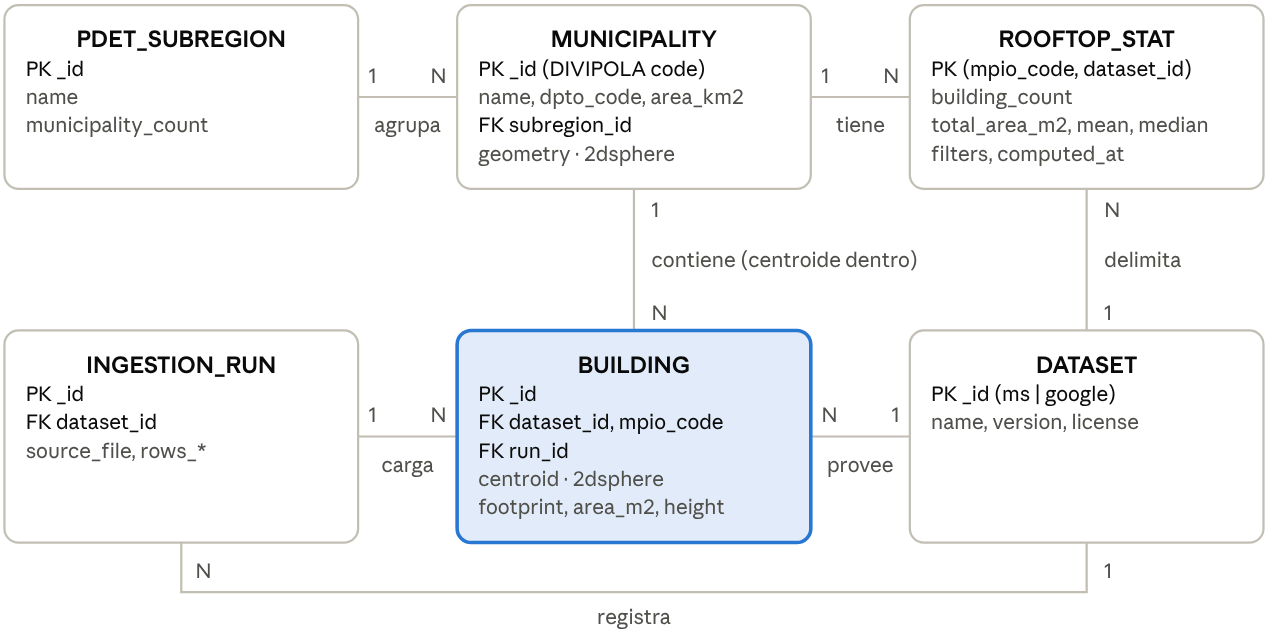


Diagrama E-R · 6 entidades, 7 relaciones uno a muchos

Cada línea se lee como "uno del lado 1, muchos del lado N". La relación MUNICIPALITY–BUILDING es espacial: el campo `mpio_code` se llena con una consulta `$geoWithin` en la Fase 3, no al momento de la carga.

4.1 Del modelo E-R a documentos

Relación	Patrón en MongoDB	Por qué
PDET_SUBREGION → MUNICIPALITY	Referencia (`subregion_id`) + copia de `subregion_name`	16 subregiones y 170 municipios; copiar el nombre evita un `$lookup` en los reportes

Relación	Patrón en MongoDB	Por qué
MUNICIPALITY → BUILDING	Referencia (`mpio_code` en cada edificación)	Millones de edificaciones no caben embebidas (límite de 16 MB por documento)
DATASET → BUILDING	Referencia (`dataset_id`)	Mantiene las dos fuentes en una sola colección, así la misma consulta las compara
MUNICIPALITY + DATASET → ROOFTOP_STAT	`_id` compuesto `{mpio_code, dataset_id}`	Un documento de resultado por municipio y fuente; volver a correr el cálculo lo sobrescribe
DATASET → INGESTION_RUN → BUILDING	Referencia (`run_id`)	Cada edificación se puede rastrear hasta el archivo y la carga que la insertó

5. Diccionario de datos

Seis colecciones componen el modelo; `buildings` es la única grande (millones de documentos), por eso referencia a los municipios en vez de ir embebida dentro de ellos. Los nombres de colecciones y campos se dejan en inglés porque así quedan en el código.

5.1 `pdet_subregions`

Campo	Tipo	Oblig.	Descripción
`_id`	int	Sí	Identificador de la subregión, columna `id` de `MunicipiosPDET.xlsx` (1 a 16)
`name`	string	Sí	Nombre de la subregión (ej. "Alto Patía y Norte del Cauca")
`municipality_count`	int	Sí	Número de municipios PDET de la subregión

5.2 `municipalities`

Campo	Tipo	Oblig.	Descripción
`_id`	string(5)	Sí	Código DIVIPOLA del municipio (`MPIO_CDPMP` en el MGN), ej. `19050`
`name`	string	Sí	Nombre del municipio (`MPIO_CNMBR`)
`dpto_code`	string(2)	Sí	Código del departamento (`DPTO_CCDGO`)
`dpto_name`	string	Sí	Nombre del departamento
`subregion_id`	int	Sí	Referencia a `pdet_subregions._id`
`subregion_name`	string	Sí	Copiado de la subregión para evitar un join en los reportes
`area_km2`	double	Sí	Área oficial (`MPIO_NAREA`)
`geometry`	GeoJSON Polygon / MultiPolygon	Sí	Límite en WGS84; con índice `2dsphere`
`source`	object	Sí	`{dataset: "MGN", version: "2025", file}`
`loaded_at`	date	Sí	Fecha y hora de carga

5.3 `datasets`

Campo	Tipo	Oblig.	Descripción
`_id`	string	Sí	`ms`, `google`
`name`	string	Sí	Nombre completo del dataset
`version`	string	Sí	Versión usada (ej. Google `v3`)
`license`	string	Sí	ODbL / CC BY-4.0
`url`	string	Sí	Fuente de descarga
`accessed_at`	date	Sí	Fecha de descarga

5.4 `buildings`

Campo	Tipo	Oblig.	Descripción
`_id`	ObjectId	Sí	Generado por MongoDB
`dataset_id`	string	Sí	Referencia a `datasets._id`
`source_ref`	string	No	Identificador original (Google: `full_plus_code`; Microsoft: archivo + fila)
`footprint`	GeoJSON Polygon	Sí	Contorno del techo en WGS84
`centroid`	GeoJSON Point	Sí	Centroide de `footprint`; con índice `2dsphere`, se usa para la asignación
`area_m2`	double	Sí	Área geodésica calculada con `pyproj.Geod`
`area_src_m2`	double	No	Área reportada por la fuente (solo Google), se guarda para validar
`confidence`	double	No	Confianza de la detección 0–1; null si la fuente no la trae
`height_m`	double	No	Altura reportada por Microsoft; null cuando es −1 o no viene
`mpio_code`	string(5)	No	Referencia a `municipalities._id`; se llena en la Fase 3
`run_id`	ObjectId	Sí	Referencia a `ingestion_runs._id`

5.5 `rooftop_stats`

Campo	Tipo	Oblig.	Descripción
`_id`	object	Sí	`{mpio_code, dataset_id}` — un documento por municipio y fuente
`building_count`	int	Sí	Edificaciones cuyo centroide está en el municipio, después de filtros
`total_area_m2`	double	Sí	Suma de `area_m2`

Campo	Tipo	Oblig.	Descripción
`mean_area_m2`	double	Sí	Área promedio de techo
`median_area_m2`	double	Sí	Mediana del área de techo
`filters`	object	Sí	`{min_area_m2, min_confidence}` aplicados
`computed_at`	date	Sí	Fecha y hora del cálculo

5.6 `ingestion_runs`

Campo	Tipo	Oblig.	Descripción
`_id`	ObjectId	Sí	Generado por MongoDB
`dataset_id`	string	Sí	Referencia a `datasets._id` (o `mgn`)
`source_file`	string	Sí	Archivo o partición cargada
`rows_read` / `rows_loaded` / `rows_rejected`	int	Sí	Contadores de la carga
`started_at` / `finished_at`	date	Sí	Duración de la carga
`notes`	string	No	Errores o advertencias

6. Índices, riesgos y repositorio

6.1 Índices planeados

Colección	Índice	Tipo	Para qué sirve
`municipalities`	`geometry`	2dsphere	Consultas espaciales contra los límites
`municipalities`	`subregion_id`	B-tree	Agrupar por subregión PDET
`buildings`	`centroid`	2dsphere	Asignación con `$geoWithin` a los municipios (consulta principal)

Colección	Índice	Tipo	Para qué sirve
`buildings`	`{dataset_id: 1, mpio_code: 1}`	Compuesto	Agregación por fuente y por municipio
`buildings`	`footprint`	2dsphere	Opcional; solo si probamos `$geoIntersects` con los polígonos completos

Indexar puntos en vez de polígonos hace el índice más pequeño y la prueba de punto en polígono más barata sobre millones de documentos.

6.2 Riesgos y mitigaciones

Riesgo	Mitigación
Los archivos de edificaciones de Colombia pesan decenas de GB	Descargar solo las particiones que tocan los rectángulos envolventes PDET; leer por bloques; descartar filas fuera del área antes de insertar
`2dsphere` rechaza polígonos inválidos o que se auto-intersectan	Repararlos con `make_valid` antes de insertar; registrar los rechazos en `ingestion_runs`
Los nombres de la lista PDET de la ART no coinciden con los del MGN	Cruzar por código DIVIPOLA, nunca por nombre
El MultiPolygon de un municipio supera los 16 MB	Revisar el tamaño del documento al cargar; simplificar con una tolerancia fija solo si hace falta, y documentarlo
Las dos fuentes difieren mucho en una zona	Reportar ambas cifras y la diferencia; nunca promediarlas sin decirlo

6.3 Estructura del repositorio

```
README.md                  cómo correr todo, en orden
docker-compose.yml         MongoDB 7
config.yaml                rutas, URLs, umbrales
requirements.txt
docs/
  delivery1/               este documento (PDF) + diagrama E-R
scripts/
  01_download_mgn.py
  02_load_municipalities.py
  03_download_buildings.py
  04_load_buildings.py
  05_eda.ipynb
  06_rooftop_stats.py
  07_maps.ipynb
data/                      descargas crudas (ignoradas por git)
outputs/                   tablas y mapas
```